



Arcanum Landing Engine — AI-генерация лендингов курсов для онлайн-школы

Реализовано: ASRP · **Продукт:** Arcanum 12th (arcanum12th.education) · **Роль:** Проектирование и разработка полного цикла (агент · API · редактор авторов · публичный рендерер) · **Тип сотрудничества:** Собственный продукт · **Домен:** AI-автоматизация контента для мультиязычной образовательной платформы

1. Краткое резюме

Arcanum Landing Engine — это **AI-система полного цикла для сборки лендингов курсов** собственной онлайн-школы ASRP **Arcanum 12th** (живой сайт arcanum12th.education, 14 курсов, 14 языков интерфейса). Она превращает структурированные данные курса в опубликованную маркетинговую страницу почти без ручной вёрстки — и при этом оставляет человека полностью управляющим результатом.

Цикл состоит из четырёх взаимодействующих частей, все созданы ASRP: **GraphQL-бэкенд** (`arcanum12th-api`) собирает «build context» из реальных данных курса и структурный контракт; **headless-AI-агент** (`agent-landing-builder`, FastAPI + OpenAI) планирует раскладку, пишет текст, валидирует по каталогу блоков и сам себя чинит — выдавая JSON-payload типизированных секций; **Nuxt-редактор авторов** (`arcanum12th-author-web`) позволяет автору дорабатывать этот payload визуально — переставлять, добавлять блоки из 🎨-палитры, редактировать каждый блок инлайн, настраивать токены темы light/dark, переключать язык; а **публичный Nuxt-рендерер** (`website`) сопоставляет тип каждой секции с Vue-компонентом и рисует живую страницу. Единица обмена по всему циклу — один **landing payload**: `{ sections: [{ id, type, props }], themeStyles }`.

Она работает в продакшне и генерирует живые лендинги курсов на arcanum12th.education.

2. Позиционирование и проблема

Онлайн-школе с растущим каталогом курсов нужен **маркетинговый лендинг на каждый курс** — hero, программа, статистика, цены, FAQ, контакты — на нескольких языках. Собирать каждый вручную медленно и качество «плывёт»; сгенерировать из одного промпта «сделай лендинг» — получить неструктурированный HTML, который никто не может безопасно редактировать или переводить.

Ответ Arcanum трактует лендинг как **структурированные данные, а не разметку**, и делит работу между машиной и человеком. Машина (агент) хорошо набрасывает полную раскладку по каталогу из фактов; человек (автор) хорош в суждении — финальные формулировки, порядок, изображения, ощущение бренда. Поскольку передача — это типизированный **payload секций**, а не HTML, один и

тот же артефакт можно сгенерировать, доработать в редакторе, перевести по языкам, сохранить, аудировать и отрендерить — каждая стадия владеет своей заботой и не разбирает заново вывод соседей.

3. Что было реализовано

Четыре взаимодействующих компонента, все созданы ASRP:

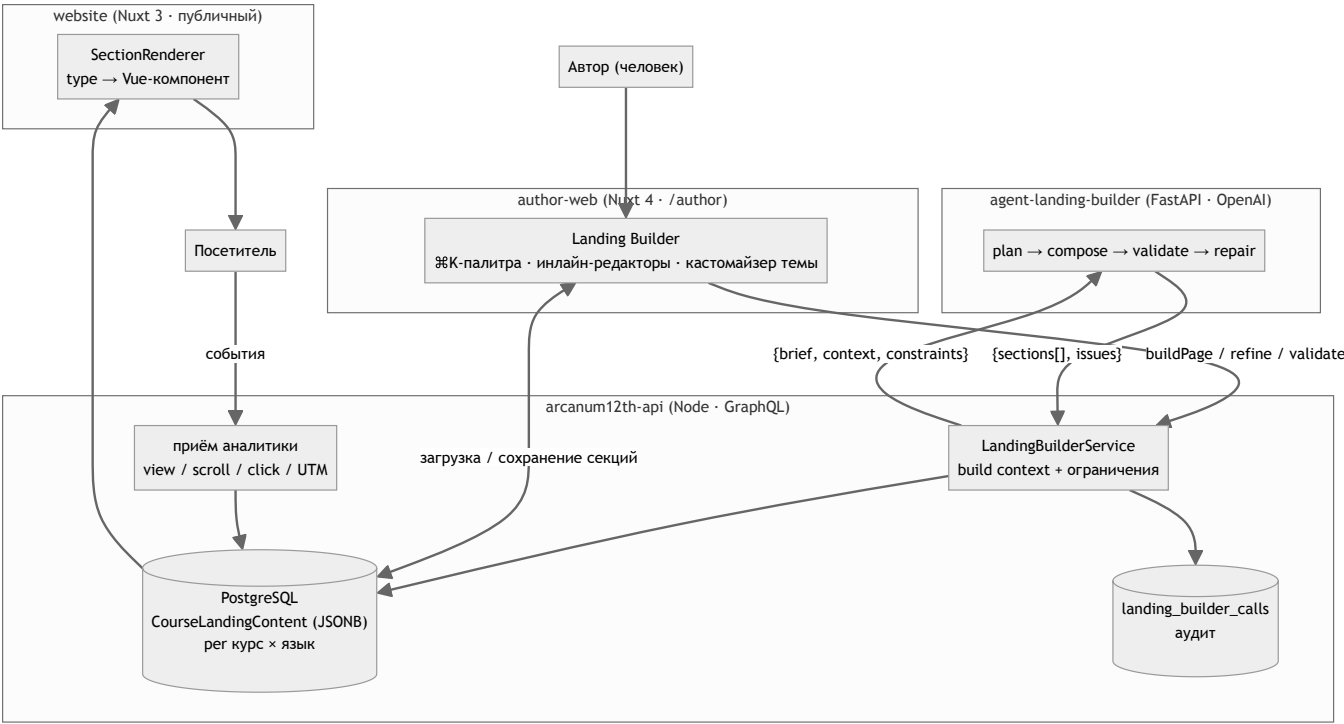
- **agent-landing-builder** — **AI-генератор**. FastAPI-микросервис (OpenAI `gpt-4o-mini`) с эндпоинтами `/build-page`, `/refine-page`, `/validate-page`. Планирует раскладку, наполняет props каждого блока текстом, валидирует по каталогу блоков и запускает ограниченный цикл починки — с детерминированным fallback на каждой LLM-стадии, поэтому работает и без API-ключа.
- **arcanum12th-api** — **бэкенд и оркестратор**. Node.js GraphQL-API (Apollo Server 4, Sequelize/PostgreSQL) с четырьмя оверлеями приложений (`public` / `admin` / `author` / `partner`). Его `LandingBuilderService` собирает build context и структурный контракт из реальных данных курса, зовёт агента, нормализует и сохраняет payload и логирует каждый вызов для аудита. Также принимает аналитику лендинга (просмотры, скроллы, клики, UTM).
- **arcanum12th-author-web** — **редактор авторов**. Nuxt 4 админ-приложение на `/author`. Помимо полного управления контентом (статьи, курсы, уроки), его флагман — **Landing Builder**: 🎨-палитра добавления секций (~60 типов), инлайн-редакторы секций, кастомайзер темы `light/dark`, переключение языков, шаблоны и импорт/экспорт JSON.
- **website** — **публичный рендерер**. Nuxt 3 сайт, читающий сохранённый payload для локали посетителя и сопоставляющий каждый `section.type` с Vue-компонентом, внедряя `themeStyles` как CSS-переменные и отправляя аналитику обратно в API.
- **Более широкая экосистема**. Та же платформа (общий GraphQL-API) также питает канал **Telegram-бот** — просмотр каталога курсов, обучающий поток, AI-дневник снов и платежи — второй потребитель тех же данных курса рядом с публичным сайтом.

[screenshot /скриншот: `assets/arcanum-home.png` — Arcanum 12th home /главная страница школы]

[screenshot /скриншот: `assets/arcanum-courses.png` — course catalog (14 courses) /каталог курсов (14 курсов)]

4. Архитектура

Четыре части образуют петлю вокруг одного payload. API — это хаб: он владеет контрактом и хранением; агент набрасывает; редактор дорабатывает; сайт рендерит.



Компонент	Зона ответственности
author-web (Nuxt 4)	UX авторинга: редактор Landing Builder, каталог блоков + дефолтные props, кастомизация темы, переключение языков и триггер «сгенерировать через AI».
арcanum12th-api (Node/GraphQL)	Хаб: собирает build context + структурный контракт, оркестрирует агента, нормализует и сохраняет payload (JSONB per курсхязык), аудит-лог, приём аналитики.
agent-landing-builder (FastAPI/OpenAI)	Генератор: планирование раскладки, копирайтинг, валидация по каталогу и ограниченная самопочинка — с детерминированными fallback.
website (Nuxt 3)	Публичный рендер: реестр type → компонент, токены темы как CSS-переменные, события аналитики.

5. Конвейер генерации

Внутри агента `build_landing()` выполняет четыре стадии, у каждой детерминированный fallback, поэтому сервис деградирует мягко, когда OpenAI недоступен (через `_should_use_llm()`):

- Planner** — выбирает упорядоченный план секций (`{family, type, required, ...}`) из каталога блоков, соблюдая ограничения API (`must_include, preferred_order, max_sections`). LLM-промпт `@landing-planner`; детерминированный fallback `plan_sections()`.
- Composer** — пишет props каждой запланированной секции из переданного контекста курса. LLM-промпт `@landing-copywriter`; детерминированный fallback `compose_payload()`.

3. **Validator** — проверяет payload по каталогу + ограничениям (`validate_payload()`), с LLM-оценщиком качества (`@landing-evaluator`) и детерминированной оценкой качества.
4. **Repair** — LLM-проход рефлексии (`@landing-reflector`) плюс детерминированный `repair_payload()` . Оркестратор **оставляет починку, только если она снижает число проблем** (`if len(repaired_issues) <= len(issues)`) — ограниченно, никогда не хуже.

Бэкенд формирует свободу агента через **layout hints**: на каждый блок — `mode llm_content` (пишет модель) или `deterministic_props` (заполняется из контекста, напр. цены, статистика), плюс источники `fill_from_context` . Так факты остаются фактами, а моделью генерируется только продающий текст.

6. Контракт лендинга и каталог блоков

Контракт — это один payload — `{ sections: [{ id, type, props }], themeStyles: { light, dark } }` — и словарь `type` секций, сгруппированных в семейства (Header, Hero, About, Features, Stats, Timeline, Steps, Blog, Pricing, FAQ, Social proof, CTA, Contact, Map, Footer и платформенные блоки вроде динамических карточек курсов). У многих семейств есть **версионированные варианты** (напр. `stats-v1..v6` , `features-v1..v6` , `timeline-v1..v4` , `faq-v1..v4`), поэтому раскладка может меняться визуально без изменения формы данных.

Честная инженерная заметка: этот контракт **реплицируется по соглашению в трёх местах** — каталог агента (`agent/catalog/defaults/**`), каталог редактора (`useLandingSections.ts` + `landingDefaultProps.ts` + `landingSectionComponents.ts`) и мапа `type` → компонент публичного рендерера (`SectionRenderer.vue`). Тип на проводе — непрозрачный JSON, поэтому ничто не проверяет его сквозной машинно. Коммит «**Sync catalog with frontend landing contracts**» (агент, 2026-06-09) — как раз момент, когда каталог агента пришили к реальным формам блоков/props фронтенда. Компромисс реален: если тип секции есть в редакторе, но нет в рендерере, публичная страница молча ничего для неё не рисует — известный риск дрейфа, присущий контракту «по соглашению».

[*screenshot /скриншот: `assets/arcanum-author-section-library.png` — `##K section/template palette` / палитра секций и шаблонов*]

7. Прогресс и итерации

Движок заметно улучшался от прогона к прогону. Папка `artifacts/` фиксирует две параллельные серии `v1→v7` (ответы сборки и извлечённые payload) за **2026-06-08** → **06-09**. Число секций держится на 12, поэтому прирост виден в **структуре и богатстве props** — число заполненных ключей `props` растёт с ~139–153 в начале до **170 (v5)** и **179 (v7)**, пустые `props` падают до единиц к середине серии, секции переставляются (`contact` выше, `FAQ` раньше), а поздние версии подставляют улучшенные варианты (`features-v2` , `faq-v1`). Это цикл починки/оценки, ужимающий вывод от прогона к прогону.

Git-история рассказывает то же как майлстоуны:

Фаза	Что появилось
Каркас и ядро пайплайна (06-06)	Первичный билдер, LLM-пайплайн планирования, тесты, curl-примеры
Ops и метаданные каталога (06-06→07)	Docker CI, roadmap качества, обогащённый каталог, семантическая валидация + golden-тесты, трассировка запросов
Генерализация контекста (06-07→08)	Приём общего/сырого контекста курса, структурированный контекст курса
Детерминизм цен и разбивка каталога (06-08)	Планирование с учётом цен, детерминированные цены, разбивка каталога на manifest-семейства
Планирование раскладки + LLM eval/reflection (06-08→09)	Абстрактный план раскладки, LLM-оценка, промпты рефлексии, positioning-memo
Чистка контракта (06-08→09)	Прекратить генерацию темы, обязательные секции от бэкенда, документированный layout-driven-контракт
Синхронизация каталога и харденинг (06-09→12)	Линтинг каталога + benchmark-харнесс, разделение planner/copywriter, синхронизация каталога с контрактами фронтенда , более строгие эвристики текста

Окружающий стек созрел в тот же период: фича лендинга в API (CourseLandingContent) появилась **2026-05-13**, а эпик Landing Builder в author-web шёл **май–июль 2026**.

8. Технологический стек

Слой	Выбор
Агент	Python 3.13, FastAPI + Uvicorn, Pydantic v2, OpenAI (gpt-4o-mini , из env, temperature 0 , JSON-режим)
Бэкенд API	Node.js, Apollo Server 4 + Express, GraphQL, Sequelize + PostgreSQL, Redis (сессии/кэш), pg-boss; авторизация JWT/API-key; четыре оверлея приложений
Редактор авторов	Nuxt 4 (Vue 3, SSR/Nitro), Pinia, shadcn-vue + Tailwind v4, Apollo Client + GraphQL codegen, TipTap v3, xstate, 14 языков
Публичный рендерер	Nuxt 3 (Vue 3), Tailwind v4, Apollo Client, i18n; реестр секций type → компонент , токены темы как CSS-переменные
Эксплуатация	Docker Compose, образы в GHCR, Nginx + certbot (TLS), PM2-кластер; submodule-монорепо (arcanum12th-core)

9. Данные и интерфейсы

- **Хранение.** `CourseLandingContent { courseId, languageId, sections: JSONB }` — **один payload на (курс, язык)**, мягкое удаление (`paranoid`). Сайт подгружает его через публичный запрос `course`; отдельного эндпоинта «получить лендинг» нет.
- **GraphQL-поверхность.** Оверлей `author: courseLandingContentBuildPage / refinePage / validatePage, courseLandingContentBuildContext` (предпросмотр собранного контекста), плюс `CRUD courseLandingContentCreate / Update / Delete`. Оверлей `public: мутации аналитики courseLandingView / Click / Scroll / Utm`. Оверлей `admin: аудит-запрос landingBuilderCalls`.
- **API агента.** `POST /build-page, POST /refine-page` (требуется `previous_draft`), `POST /validate-page, GET /health`.
- **Аудит.** Каждый вызов агента пишется в `landing_builder_calls` (запрос, ответ, проблемы, контекст, тайминги, статус) — полный след генерации.

10. Надёжность и эксплуатация

- **Мягкая деградация.** У каждой LLM-стадии есть детерминированный `fallback`; без ключа OpenAI агент всё равно выдаёт валидный `payload` по каталогу.
- **Ограниченная починка.** Починка сохраняется, только если снижает число проблем — цикл не может ухудшить страницу.
- **Человек в контуре.** Вывод AI — это *черновик*; ничто не публикуется, пока автор не сохранит в редакторе.
- **Аудируемость.** `landing_builder_calls` фиксирует каждую генерацию; сквозная трассировка `X-Request-Id` идёт через агента.
- **Аналитика.** Просмотры, скроллы, клики и UTM летят с публичной страницы обратно в API.
- **Развёртывание.** Docker Compose за Nginx + certbot TLS; образы в GHCR; PM2-кластер внутри контейнеров; Postgres 17 + Redis. Агент — внутренний сервис, от которого зависит `author-API`. Стек собран как submodule-монорепо с `manifest.json`, пинящим SHA каждого сабмодуля.

11. Посмотреть вживую

- **Живой продукт:** arcanum12th.education — школа, 14 курсов, несколько локалей. Лендинги курсов — это отрендеренный вывод этого движка.
- **Панель автора:** arcanum12th.education/author — редактор Landing Builder и более широкая авторская платформа (богатый Markdown-редактор, мульти-формат статьи/курсы/уроки, ИИ-ассистент, 500+ авторов).

Скриншоты ниже показывают авторскую сторону (доработку AI-черновика `payload` секций) и отрендеренные публичные лендинги.

(EN captions above apply to both languages.)

12. Статус и планы

Статус: продакшн. API, author-web и публичный сайт развёрнуты и работают на arcanum12th.education по 14 курсам. Агент — хорошо структурированный движок версии 0.1.0 : сильное покрытие тестами (API, LLM-пайплайн, live smoke, семантическая валидация, golden-брифы, линт каталога, харнесс оценки), CI Docker-сборки и трассировка запросов; харденинг интеграции (auth/rate-limiting на эндпоинтах агента) ещё в планах. Естественные следующие шаги: единый общий пакет каталога, чтобы убрать трёхсторонний дрейф контракта, схемы props по типам (сейчас props свободной формы) и закрытие расхождения вариантов редактор↔рендерер.

13. Что реализовала ASRP

ASRP спроектировала и построила всю систему целиком — все четыре компонента (agent-landing-builder , фичу лендинга в arcanum12th-api + LandingBuilderService , Landing Builder в arcanum12th-author-web и рендерер website), общий контракт landing-payload и каталог блоков, топологию развёртывания и конвейер AI-генерации — как собственный продукт для собственной онлайн-школы.

Кейс-стади собственного продукта, подготовленный ASRP для портфолио. Arcanum 12th и arcanum12th.education — собственный продукт ASRP и названы соответственно. Цифры отражают систему по состоянию на 2026-07; публичный сайт работает. Исходные репозитории приватны; приведены только детали уровня возможностей.